

1. Contextualización

El desafío consiste en implementar el juego Sweet Crush, un juego de emparejamiento donde el jugador elimina fichas que forman secuencias de tres o más iguales en horizontal o vertical, generando reorganizaciones (cascadas) del tablero. Sin embargo, el objetivo real de la actividad no es el juego en sí, sino evaluar el dominio y lo aprendido en clase de conceptos fundamentales de C++: manipulación de bits, gestión de memoria dinámica y programación estructurada. La especialidad que distingue este desafío de una implementación convencional es la restricción de representar cada ficha del tablero en exactamente 3 bits, sin desperdiciar espacio ni alinear a byte, lo que obliga a trabajar directamente con operadores bitwise (&, |, ^, ~, <<, >>) en cada acceso o modificación del tablero, en lugar de usar estructuras de datos convencionales.

Estas restricciones no buscan únicamente dificultar la implementación, sino evaluar destrezas específicas que suelen quedar ocultas cuando se usan librerías de alto nivel. Prohibir STL, struct y class obliga al estudiante a manejar explícitamente el estado del programa a través de parámetros y punteros, en lugar de delegarlo a contenedores que resuelven la gestión de memoria automáticamente; esto evidencia si el estudiante realmente comprende cómo se organiza la memoria detrás de ese planteamiento. Prohibir el desempaquetado del tablero a una matriz auxiliar (por ejemplo, un arreglo de enteros donde cada posición ocupe una ficha) impide que el usuario pueda "hacer trampa" evitando la manipulación de bits real: sin esta restricción, el operador podría cumplir superficialmente el requisito de tener el tablero en 3 bits en algún punto, pero trabajar internamente sobre una copia expandida, sin ejercitar el razonamiento bitwise que el desafío busca evaluar. Finalmente, exigir que la memoria reservada corresponda exactamente al tamaño lógico del tablero (evitando sobredimensionar desde el inicio) previene la práctica común de estudiantes de "resolver" la gestión dinámica de memoria reservando de más para no tener que lidiar con redimensionamientos reales, lo cual anula el propósito de evaluar el manejo correcto de new/delete."

2. Análisis del problema

2.1 RESTRICCIONES Y TÉCNICAS

A partir del documento del desafío se identificaron las siguientes restricciones como determinantes para el diseño de la solución:

- **Cero POO / STL / Structs:** prohibido usar class, struct, template, std::string o contenedores de la STL (vector, list, etc.).
- **Cero librerías externas de tipo:** solo tipos primitivos de C++ y arreglos/punteros dinámicos (unsigned char*).
- **Representación compacta (3 bits/ficha):** las fichas miden 3 bits y se empaquetan de forma continua en memoria; los bits de una ficha pueden quedar partidos entre dos bytes.
- **Manipulación de bits obligatoria:** toda lectura, escritura o modificación debe hacerse con operadores bitwise; prohibido desempaquetar el tablero a una matriz auxiliar.
- **Gestión dinámica de memoria exacta:** la memoria asignada debe responder al tamaño real del tablero; prohibido apartar matrices fijas o sobredimensionadas.

- **Regla del 65% para reducción:** al eliminar filas/columnas, la memoria física solo se reduce cuando la memoria utilizada cae por debajo del 65% del tamaño actualmente reservado.
- **Alineación de bits limpios:** las tramas de bits válidas inician en el bit menos significativo; los bits sobrantes o inválidos deben quedar agrupados al final.
- **Lógica de juego completa:** eliminaciones, combinaciones mayores e iguales a 3, gravedad, rellenado, detección automática de cascadas y puntuación.
- **Modificación estructural intermedia:** permite insertar o eliminar filas y columnas en cualquier posición, no solo en los extremos.

2.2 Impacto de las restricciones sobre el diseño

- **Cero POO/STL/Structs** Obliga a recargar las firmas de las funciones con múltiples parámetros por referencia. Como no hay un objeto o estructura para empaquetar los datos, el estado del programa debe de estar en la función principal y debe de ser usado manualmente en el módulo mediante paso por referencia.
- **Representación compacta (3 bits/ficha) + manipulación bitwise obligatoria** Obliga a crear una capa de abstracción con dos funciones primitivas de lectura y escritura de una ficha. Es necesaria esta lógica porque, si se intentan manipular los desplazamientos de bits repartidos entre bytes directamente dentro de la lógica del juego, el código se vuelve difícil de mantener y depurar.
- **Gestión dinámica de memoria exacta + regla del 65%** Separa la dimensión lógica del tablero (filas $\times$ columnas) de la dimensión física reservada en memoria, llevando dos valores en paralelo: los bytes que realmente se necesitan y los bytes efectivamente reservados. Al eliminar elementos, esto condiciona la liberación física del arreglo solo cuando el uso real cae por debajo del 65%, evitando pedir y liberar memoria del heap de forma innecesaria en cada operación.

2.3 Propósito pedagógico de las restricciones

Estas restricciones no buscan únicamente dificultar la implementación, sino evaluar destrezas específicas que suelen quedar ocultas cuando se usan librerías de alto nivel:

- Prohibir STL, struct y class obliga al estudiante a manejar explícitamente el estado del programa a través de parámetros y punteros.
- Prohibir el desempaqueo del tablero a una matriz auxiliar.
- Exigir que la memoria reservada corresponda exactamente al tamaño lógico del tablero.

Estas restricciones buscan enseñar la importancia de mantener la eficiencia al programar, enseñar la programación a nivel de bits, y evaluar que tanto ha entendido el estudiante para codificar en C++ apoyándose únicamente en razonamiento propio y con material enviado por el docente.

3. Diseño implementado por los estudiantes

3.1 Esquema de módulos

La solución se divide en cuatro módulos, organizados en capas con dependencia unidireccional (cada módulo solo depende de los módulos de nivel inferior):

Módulo 1: Acceso a Bits y Gestión Dinámica de Memoria.

Módulo 2: Operaciones Estructurales y Lógica del Tablero.

Módulo 3: Reglas del Juego y Cascadas Este módulo también concentra el estado del juego (puntuación, eliminaciones del usuario, fichas eliminadas totales, combinaciones detectadas, cascadas de la eliminación actual).

Módulo 4: Interfaz de Usuario y Control.

3.2 Orden del uso de los módulos

Módulo 4 (Interfaz/Control); Módulo 3 (Reglas del Juego + Estado); Módulo 2 (Operaciones Estructurales); Módulo 1 (Acceso a Bits + Memoria).

1. Módulo 4 → llama a Módulo 3

- Función: ejecutarCicloCascada(...)
- Parámetros: unsigned char* &buffer, int &filas, int &cols, int &bytesReservados, int &puntuacion, int &cascadasActuales, int &fichasEliminadasTotal.
- Razón: el Módulo 3 necesita conocer y eventualmente modificar las dimensiones del tablero, evaluar si la memoria requiere reducción, y actualizar los contadores de puntuación y cascadas.

2. Módulo 3 → llama a Módulo 2

- Función: aplicarGravedad(...) / rellenarEspaciosVacios(...)
- Parámetros: unsigned char* buffer, int filas, int cols.
- Razón: el Módulo 2 solo reacomoda fichas dentro del espacio ya existente (no cambia el tamaño del tablero), por lo que no necesita conocer ni alterar los contadores de puntuación ni la capacidad de memoria reservada.

3. Módulo 2 → llama a Módulo 1

- Funciones: obtenerFicha(...), establecerFicha(...)
- Parámetros: unsigned char* buffer (o const unsigned char* para lectura), int fila, int col, int cols, unsigned char valor.
- Razón: el Módulo 1 realiza únicamente la micro-operación bitwise (&, |, <<, >>) sobre la posición calculada (índice × 3 bits), sin necesitar ningún otro contexto del juego.